

CSE 220 -- DFT / FFT Practice Question Bank

Companion practice set for the Fourier Hybrid Image lab (A1/A2)

Generated study material -- 10 independent problems, each sized for a 40-45 minute closed-lab test

How to use this document

Each problem below is written in the same format as the real lab sheet: a short mathematical model, 3 TODO-sized code blocks, a fixed mark allocation (10 marks total), a run command, and a verification rule. Every problem reuses the infrastructure you already built -- **transforms.py** (DFTAnalyzer / FFTTransformer / ArbitraryLengthFFT), **image_conv.py** (transform_2d / inverse_2d / convolve_image), and **image_utils.py** (load_image / save_image / make_kernel) -- exactly as `hybrid_image_template.py` does. Do not modify those provided files in the real test; only fill in the TODO functions. On the day of the test you will most likely receive ONE of these problems (or something close to it), not all ten.

The matching solving scaffold is **practice_template.py**, provided separately. It contains one runnable section per problem below (same function names, same TODO markers) so you can time yourself against each one tonight.

Every problem here is intentionally **harder** than the original hybrid-image lab: most combine two spectra-manipulation ideas instead of one, several require you to derive the padding/cropping rule yourself instead of being told it, and two require picking between DFT/FFT/Bluestein engines correctly for a non-power-of-two case.

Problem 1 -- High-Boost Sharpening in the Frequency Domain

Marks: 10 | Suggested time: 40 minutes

High-boost sharpening exaggerates detail by adding back a scaled copy of the high-frequency residual: given a low-pass kernel spectrum G and boost factor $A \geq 1$, the sharpened spectrum is

$$Y = X \cdot G + A \cdot X \cdot (\Delta - G) = X \cdot (A \cdot \Delta - (A-1) \cdot G)$$

where X is the padded spectrum of the source image and Δ is the centred-impulse spectrum (provided, same helper as the lab). Complete **sharpen_plane**, **choose_transform_shape** (reuse), and **sharpen_image** in the template.

- **choose_transform_shape(image_shape, kernel_shape, engine)** -- identical contract to the lab. [1]
- **sharpen_plane(plane, kernel, boost, engine)** -- build Y from a single spectral combination as shown above, inverse-transform once, take the real part, and crop back to the original plane shape using the same kernel-centre offset rule as the lab. [6]
- **sharpen_image(image, kernel, boost, engine)** -- dispatch grayscale or RGB, preserving shape. [3]

Run:

```
python3 practice_template.py p1 --image images/skyline512.png \
--kernel-size 15 --boost 2.5 --engine fft --out-dir outputs/p1
```

Verification. Compare against the independent oracle `plane + boost*(plane - convolve_image(plane, kernel, engine))`; max absolute error must be $\leq 1e-9$, or MISMATCH.

Trap: $A \cdot \Delta$ is only a scalar multiple of an impulse spectrum -- do not recompute Δ per boost value inside a loop; compute it once per call like the lab does.

Problem 2 -- Band-Pass Filtering with a Ring-Shaped Mask

Marks: 10 | Suggested time: 45 minutes

Instead of combining two images, this problem keeps only a frequency *annulus* of a single image: given cutoffs $r_{\text{low}} < r_{\text{high}}$ (in cycles across the padded transform), zero out every frequency bin whose radius from DC lies outside $[r_{\text{low}}, r_{\text{high}}]$.

- **frequency_grid(shape)** -- return the 2D array of shifted radial frequency magnitudes (0 at the corners *before* any fftshift-style reordering -- you must build the radius map directly from unshifted index arithmetic, since `numpy.fft.fftshift` is banned). [3]
- **bandpass_mask(shape, r_low, r_high)** -- boolean array, True where $r_{\text{low}} \leq \text{radius} \leq r_{\text{high}}$. [2]
- **bandpass_plane(plane, r_low, r_high, engine)** -- transform, apply the mask pointwise, inverse transform, return the real part cropped back to the input shape (no kernel padding is needed here, only power-of-two padding when `engine.name == 'fft'`). [5]

Run:

```
python3 practice_template.py p2 --image images/sunset512.png \
--r-low 8 --r-high 40 --engine dft --out-dir outputs/p2
```

Verification. Report the fraction of spectrum energy retained ($\sum |Y|^2 / \sum |X|^2$) and confirm it is strictly between 0 and 1; also confirm that running with $r_{\text{low}}=0$, $r_{\text{high}}=\text{infinity}$ reproduces the original image to within $1e-9$.

Trap: your radius map must be built with the same 'negative frequencies wrap to the top half of the index range' convention the DFT itself uses -- i.e. frequency index k corresponds to k for $k \leq N/2$ and to $k-N$ for $k > N/2$. Getting this wrong silently produces a mask that looks fine on a picture of the mask, but corrupts the filtered image.

Problem 3 -- Non-Blind Wiener Deconvolution (Deblurring)

Marks: 10 | Suggested time: 45 minutes

Given an already-blurred image $B = K * S$ (convolution of a sharp image S with a known kernel K) plus noise, Wiener deconvolution recovers an estimate of S without ever dividing by a spectrum value that could be zero:

$$\hat{S}(f) = B(f) \cdot \text{conj}(K(f)) / (|K(f)|^2 + \lambda)$$

- **choose_transform_shape** -- reuse the lab's linear-convolution rule. [1]
- **wiener_deconvolve_plane(blurred, kernel, lam, engine)** -- pad, transform both, apply the formula above pointwise, inverse-transform once, take the real part, and undo the same kernel-centre crop offset used when the image was originally blurred. [6]
- **wiener_deconvolve_image(blurred, kernel, lam, engine)** -- grayscale/RGB dispatch. [3]

Run:

```
python3 practice_template.py p3 --blurred outputs/p3/blurred_input.png \
--kernel-size 21 --lam 1e-4 --engine fft --out-dir outputs/p3
```

Verification. The template pre-blurs a known source image with `convolve_image` and a fixed seed of Gaussian noise, then discards the source. Your recovered image must have $\text{RMSE} \leq$ a printed threshold against the (secretly retained) original when `lam` matches the noise level used -- report PASS/FAIL, and always print RMSE regardless of outcome.

Trap: $\lambda = 0$ exactly reduces to ordinary inverse filtering and is numerically unstable wherever $K(f)$ is small -- your code must still run (no divide-by-zero crash) because the harness always tests `lam=0` as one of its three checks.

Problem 4 -- Frequency-Domain Template Matching (Normalized Cross-Correlation)

Marks: 10 | Suggested time: 45 minutes

Cross-correlation of image I with template T can be computed as a convolution with a flipped, conjugated template: $\text{corr} = \text{IFFT}(\text{FFT}(I) \cdot \text{conj}(\text{FFT}(T)))$, evaluated at the linear (non-circular) size so a match near the border is not corrupted by wraparound.

- **choose_transform_shape(image_shape, template_shape, engine)** -- same linear-size + power-of-two rule as the lab, but here the two operands are the full image and the (usually much smaller) template rather than image and kernel. [2]
- **cross_correlate_plane(image_plane, template_plane, engine)** -- build the correlation surface with exactly one forward transform per operand and one inverse transform, and return only the valid region -- the $(R-K_r+1) \times (C-K_c+1)$ block where the template fully overlaps the image (a stricter crop than the lab's 'same-size' crop). [6]
- **best_match_location(corr_surface)** -- return the (row, col) of the maximum correlation value as a plain Python tuple of ints. [2]

Run:

```
python3 practice_template.py p4 --image images/skyline512.png \
--template outputs/p4/patch.png --engine fft --out-dir outputs/p4
```

Verification. The template is cut from a known location inside the source image before you run; your `best_match_location` must equal that location exactly, and the harness also checks your correlation surface against a direct (schoolbook, $O(R \cdot C \cdot K_r \cdot K_c)$) correlation on a small 64x64 crop to 1e-6 tolerance.

Trap: correlation is convolution with the template flipped in both axes AND conjugated -- if you forget the conjugate/flip you will get the auto-correlation-shaped result shifted to the wrong corner, and it will still 'look plausible' on a symmetric template.

Problem 5 -- Two-Kernel Multi-Focus Fusion (Three-Way Spectral Blend)

Marks: 10 | Suggested time: 45 minutes

Generalize the lab's two-source hybrid to three sources with two independent kernels G_1 , G_2 (G_1 narrower than G_2). Source A supplies content below G_1 's cutoff, source B supplies the band between G_1 and G_2 , and source C supplies everything above G_2 :

$$Y = A \cdot G_1 + B \cdot (G_2 - G_1) + C \cdot (\Delta - G_2)$$

- **choose_transform_shape(image_shape, kernel_shapes, engine)** -- kernel_shapes is now a list of the two (possibly different-sized) kernels; pad for the largest one. [2]
- **trifocal_plane(a_plane, b_plane, c_plane, kernel1, kernel2, engine)** -- build Y with exactly one inverse transform and crop using the LARGER kernel's centre offset (this is the part students usually get wrong when kernel1 and kernel2 differ in size). [6]
- **trifocal_image(...)** -- grayscale/RGB dispatch, same pattern as hybrid_image. [2]

Run:

```
python3 practice_template.py p5 --near images/skyline512.png \  
--mid images/sunset512.png --far images/skyline512.png \  
--kernel1-size 9 --kernel2-size 31 --engine fft --out-dir outputs/p5
```

Verification. Independent oracle: $\text{low} = \text{convolve}(A, G_1)$; $\text{mid} = \text{convolve}(B, G_2) - \text{convolve}(B, G_1)$; $\text{high} = C - \text{convolve}(C, G_2)$; $\text{reference} = \text{low} + \text{mid} + \text{high}$. Max abs error vs your result must be $\leq 1e-9$.

Trap: G_1 and G_2 must be zero-padded to the SAME transform shape (the one sized for the larger kernel) before either is transformed, or the pointwise multiply in the band term will silently operate on mismatched array sizes / wrong-length spectra.

Problem 6 -- Sub-Pixel Image Shifting via the Fourier Shift Theorem

Marks: 10 | Suggested time: 40 minutes

The shift theorem states that shifting a signal by (dr, dc) pixels (dr, dc need not be integers) multiplies its spectrum by a linear phase ramp:

$$Y[u, v] = X[u, v] \cdot \exp(-2j\pi(u \cdot dr/R + v \cdot dc/C))$$

- **shift_phase_ramp(shape, dr, dc)** -- build the exp(...) array above directly from the DFT's own frequency-index convention (same wrap-around convention as Problem 2 -- indices > N/2 represent negative frequencies). [4]
- **shift_plane(plane, dr, dc, engine)** -- transform the UN-padded plane at its native size (no linear-convolution padding needed for a pure shift), multiply by the ramp, inverse transform, return the real part at the original shape. [4]
- **shift_image(image, dr, dc, engine)** -- grayscale/RGB dispatch. [2]

Run:

```
python3 practice_template.py p6 --image images/sunset512.png \
--dr 13.5 --dc -6.25 --engine fft --out-dir outputs/p6
```

Verification. For integer dr, dc the result must equal `np.roll(image, (dr, dc), axis=(0,1))` to 1e-9. For non-integer dr, dc, the harness checks that `shift_plane(shift_plane(x, dr, dc), -dr, -dc)` reconstructs x to 1e-6 (round-trip test, since there is no simple oracle for a fractional shift alone).

Trap: this is the one problem in the set where padding to a linear-convolution size is WRONG -- a pure shift is a circular operation, so padding to R+Kr-1 style dimensions changes the answer. Only power-of-two padding (for the FFT engine) is legitimate here, and even that changes the periodic wrap point, so prefer running this one with the DFT engine unless told otherwise.

Problem 7 -- Circular vs. Linear Convolution: Aliasing Measurement

Marks: 10 | Suggested time: 40 minutes

This problem asks you to deliberately under-pad a convolution to a circular length and quantify the resulting wraparound error, then show the correctly padded result alongside it.

- **circular_convolve_plane(plane, kernel, engine)** -- convolve at transform length equal to $\max(\text{plane.shape}, \text{kernel.shape})$ in each axis (i.e. NO linear-convolution padding), producing a circularly-wrapped result cropped to the plane's own shape. [3]
- **linear_convolve_plane(plane, kernel, engine)** -- the lab's correct version (reuse `choose_transform_shape` and the centred-crop rule). [3]
- **aliasing_error_map(plane, kernel, engine)** -- return $|\text{circular_convolve_plane}(\dots) - \text{linear_convolve_plane}(\dots)|$ as a 2D array, plus print the fraction of pixels whose error exceeds $1e-3$. [4]

Run:

```
python3 practice_template.py p7 --image images/skyline512.png \
--kernel-size 25 --engine fft --out-dir outputs/p7
```

Verification. The harness checks that error is concentrated within $\text{kernel_size} // 2$ pixels of each border (wraparound only corrupts border pixels for a spatially compact kernel) and that `linear_convolve_plane` matches `convolve_image` from `image_conv.py` (the offline oracle) to $1e-9$ everywhere.

Trap: 'no padding' does not mean transform length equal to `plane.shape` only -- if the kernel is larger than the plane in either axis you must still pad up to the kernel's size in that axis, or the pointwise multiply shapes will not even match.

Problem 8 -- Non-Power-of-Two Hybrid Image via Bluestein / ArbitraryLengthFFT

Marks: 10 | Suggested time: 45 minutes

Re-derive the lab's hybrid-image pipeline for an odd, non-power-of-two image size (e.g. 509 x 517) using ONLY the ArbitraryLengthFFT engine, so that no power-of-two padding is applied at all -- the transform shape should be exactly the minimum linear-convolution size.

- **choose_transform_shape_arbitrary(image_shape, kernel_shape)** -- return (R+Kr-1, C+Kc-1) with NO rounding up to a power of two, regardless of engine. [2]
- **hybrid_plane_arbitrary(low_plane, high_plane, kernel, engine)** -- identical spectral combination to the lab's hybrid_plane, but must raise a ValueError immediately if engine.name != 'arbitrary' (a defensive check the lab version does not need). [5]
- **hybrid_image_arbitrary(low_image, high_image, kernel, engine)** -- dispatch, same pattern as the lab. [3]

Run:

```
python3 practice_template.py p8 --low-image images/sunset509x517.png \  
--high-image images/skyline509x517.png --kernel-size 17 \  
--engine arbitrary --out-dir outputs/p8
```

Verification. Same MATCH/MISMATCH oracle as the lab (compare against convolve_image-based low+high components), tolerance 1e-6 (looser than the lab's 1e-9 because Bluestein accumulates more rounding error than radix-2).

Trap: ArbitraryLengthFFT.transform still requires its OWN internal padding to a power of two for the chirp convolution (that machinery is already inside transforms.py) -- you must not add a second layer of power-of-two padding in choose_transform_shape_arbitrary on top of that, or the crop offsets used when reading back the result will be wrong.

Problem 9 -- Blind Spectral Watermark Embedding and Detection

Marks: 10 | Suggested time: 45 minutes

Embed a low-amplitude sinusoidal watermark directly in the frequency domain at a chosen (u_0, v_0) bin (and its conjugate-symmetric partner, so the spatial result stays real), then detect it back out of a possibly-JPEG-style-noisy copy.

- **embed_watermark_plane(plane, u0, v0, amplitude, engine)** -- transform the plane at its native size, add amplitude to bin (u_0, v_0) and amplitude (conjugated) to bin $((-u_0) \bmod R, (-v_0) \bmod C)$, inverse transform, return the real part. [5]
- **detect_watermark_plane(plane, u0, v0, engine)** -- transform the (possibly attacked) plane and return $|X[u_0, v_0]|$ as the detection statistic. [3]
- **watermark_image(image, u0, v0, amplitude, engine)** -- grayscale/RGB dispatch for embedding. [2]

Run:

```
python3 practice_template.py p9 --image images/sunset512.png \  
--u0 30 --v0 47 --amplitude 4.0 --engine dft --out-dir outputs/p9
```

Verification. detect_watermark_plane on the watermarked image must report a statistic at least 5x larger than on the un-watermarked original at the same (u_0, v_0) ; the harness also adds Gaussian noise (fixed seed) to the watermarked image and checks the statistic still exceeds the un-watermarked baseline by at least 2x.

Trap: forgetting the conjugate-symmetric partner bin makes the inverse transform's imaginary part nonzero -- your real-part crop will then silently discard the watermark's intended effect on roughly half of its energy, and the detector will under-report.

Problem 10 -- Runtime Study: Bluestein vs. Radix-2 vs. Naive DFT on 2D Convolution

Marks: 10 | Suggested time: 45 minutes

Extend the lab's hybrid-image pipeline into a benchmark comparable to bigmul.py's, but for 2D image convolution across three engines, and show the FFT curve is $O(N \log N)$ per axis while confirming Bluestein tracks it (with a larger constant) rather than degrading to $O(N^2)$ on prime sizes.

- **time_convolution(image_size, kernel_size, engine_name)** -- build a random plane of shape (image_size, image_size), a Gaussian kernel of kernel_size, run convolve_image once via bench_utils.time_best with repeats=2, and return the best time. [4]
- **run_2d_benchmark(out_dir)** -- sweep image_size over at least 5 values for each of 'dft', 'fft', and 'arbitrary' (choose sizes so 'arbitrary' can legally use non-power-of-two inputs, e.g. include a prime size like 257), stopping a sweep once a measurement exceeds TIME_BUDGET, then call plot_runtime_curve with references=('n2', 'nlogn') exactly like bench_utils is used in bigmul.py. [6]

Run:

```
python3 practice_template.py p10 --out-dir outputs/p10
```

Verification. There is no MATCH/MISMATCH here -- the deliverable is runtime_2dconv.png plus a report.txt timing table (reuse timing_table_lines). The harness inspects that the FFT curve's measured slope on a log-log plot is closer to the $O(n \log n)$ reference line than to the $O(n^2)$ reference line for the largest three points.

Trap: 2D convolution cost scales with the transform PLANE, i.e. roughly $(R*C)*\log(R*C)$ for FFT/Bluestein and $(R*C)^2$ for naive DFT -- do not reuse the 1D reference-curve exponents from bigmul.py without checking that this quadratic-in-pixels relationship is what you are actually plotting against.

Summary Table

- P1** -- High-boost sharpening (single spectral combination)
- P2** -- Ring band-pass mask (manual radial frequency index map)
- P3** -- Wiener deconvolution (division-safe spectral formula)
- P4** -- FFT template matching (conjugate/flip + valid-region crop)
- P5** -- Three-way multi-focus fusion (two differently-sized kernels)
- P6** -- Fourier shift theorem (phase ramp, no linear-conv padding)
- P7** -- Circular vs linear aliasing (quantifying wraparound error)
- P8** -- Bluestein-only hybrid image (non-power-of-two dimensions)
- P9** -- Frequency-domain watermark (conjugate-symmetric bin pair)
- P10** -- 2D convolution runtime study (three-engine benchmark + plot)

Reminder: in the real test you will not be told which single problem you receive until you open it, and (as in the reference lab) you may only touch the TODO-marked functions -- `transforms.py`, `image_conv.py`, `image_utils.py`, `io_utils.py` and `bench_utils.py` stay untouched.